

Concordia University

Team Project 5

Vehicle Rental Management System (VRMS)

CEWP MOD4 GRA/GRB (2262) Java Programming

Fall 2026 | Version 2.0

Instructional team project and presentation | September 22, 2026

Project Purpose and Scenario

Design a console application that manages vehicles, customers, availability, pricing policies, rentals, returns, maintenance, and feedback.

Rental systems combine polymorphic assets and pricing with availability conflicts, historical records, maintenance eligibility, and competing requests. The domain is well suited to data integrity and concurrency testing.

Successful outcome. A customer can find an eligible vehicle, obtain a transparent quote, create and close a valid rental, submit feedback, and view history while the system prevents double booking and preserves maintenance and availability rules.

Design authority. The model below is a starting point, not a completed solution. Your team may rename, combine, or add classes when the revised design still satisfies every required behavior and module-evidence requirement and the team can justify the decision.

Required Functional Scope

1. Load and validate vehicle, customer, and rental data from text files.
2. Add and update vehicles and maintenance records while preserving unique IDs.
3. Search eligible vehicles and calculate quotes through interchangeable pricing policies.
4. Rent and return vehicles while preventing double booking and invalid return states.
5. Record feedback only for completed rentals and retain immutable rental history.
6. Generate fleet, utilization, maintenance-due, revenue, and feedback reports and save state.

Core Domain Invariants

- Every persistent domain object has one validated, stable, unique identifier.
- A failed operation leaves the system in the same valid state that existed before the request.
- A lifecycle transition is allowed only from a documented current state to a documented next state.
- Derived totals and reports are calculated from authoritative domain state rather than duplicated mutable fields.
- Every started worker has a normal completion path, an interruption path, and bounded coordination by its owner.

Suggested Responsibility Model

Element	Kind	Primary responsibility	Relationship
Vehicle	Abstract class	ID, model, mileage, status, rate data	Parent of Car, Bike, Truck
Vehicle types	Concrete classes	Subtype capacity and eligibility rules	Extend Vehicle
Rentable	Interface	Rental and return contract	Implemented by eligible vehicle types
PricingPolicy	Interface	Produces a validated quote	Implemented by Hourly, Daily, Weekly policies
Rental	Class	Customer, vehicle, period, quote, status	Composes inner PriceBreakdown or RentalEvent
MaintenanceRecord	Class	Service dates, mileage, issues	Associated with Vehicle
Fleet	Iterable aggregate	Owns vehicle traversal	Implements Iterable of Vehicle
Repository	Generic class	Reusable vehicle or rental storage	Provides typed lookup
RentalService	Service	Quote, rent, return, maintenance validation	Coordinates transaction invariants
RentalException	Custom exception	Invalid period, state, quote, rent, or return	Thrown before partial mutation

Module 1 through Module 9 Evidence

Module	Approved focus	Project evidence
1	OOP review and Scanner	Use encapsulated model objects, a validated console menu, and Scanner-based loading of vehicles.txt, customers.txt, rentals.txt.
2	Inner classes and exceptions	Use the justified inner or nested type named in the model and custom exception handling for invalid domain transitions.
3	Comparable and Comparator	Vehicle natural order by vehicle ID; comparators by rate and mileage or availability with deterministic ties. Test comparison contracts and equal-primary-key cases.
4	Collections Framework	List for fleet and rental history, Map for stable vehicle lookup, and Set for unique active rental or maintenance identifiers. Justify each choice and prevent external mutation.
5	Iterable and Iterator	Fleet implements Iterable<Vehicle>; use an explicit Iterator for one maintenance or eligibility inspection.
6	Generics and wildcards	Repository<T extends Identifiable> or Quote<T extends PricingPolicy>; include a generic method and justified wildcard use. Do not use raw types.
7	Lambdas and Stream API	Use Predicate and Stream pipelines for eligible vehicles, maintenance-due items, utilization, or revenue summaries. Explain the source, intermediate operations, terminal operation, and empty case.
8	Reflection and annotations	Use @PricingOption or @AuditedAction and reflection to list approved public pricing or service operations. Document performance and security limits.
9	Concurrency	Simulate two customers requesting the same vehicle; synchronize the complete availability-check-and-rent transition, join workers, and prove at most one success. Use stable post-join invariants as proof.

Required Design Evidence

Problem Decomposition

Before implementation, identify the application boundary, actors, core entities, services, files, lifecycle states, shared mutable state, and stable correctness evidence. State at least three domain invariants and show which class owns each one.

UML Class Diagram

Show packages, important attributes and methods, visibility, inheritance, interface implementation, composition, the iterable aggregate, generic type relationships, custom exception, annotation, and the class that owns each Thread or Runnable lifecycle.

Sequence Diagram

Model this required workflow: Quote and rent an eligible vehicle or reject a conflicting request. Include input validation, service coordination, successful state changes, the exception path, and any worker start, join, interrupt, or synchronized operation involved.

Concurrency State View

Identify every worker, its task, the object that constructs and starts it, its termination condition, the state shared with other threads, the synchronized boundary, the join point, and the evidence read after join. Do not claim that a sampled intermediate Thread state or exact print order proves correctness.

Recommended Package Structure

Package	Responsibility
app	Application entry point and menu orchestration
model	Entities, immutable value objects, inner or nested domain types
contract	Domain interfaces and generic contracts
repository	Generic storage, file loading, and saving responsibilities
service	Use cases, validation, lifecycle transitions, and transaction boundaries
ordering	Named Comparator implementations or factories
report	Iterator, lambda, Stream, annotation, and reflection based reporting
concurrent	Runnable or Thread tasks and narrow synchronized shared-state owners
exception	Project-specific exceptions
test	Executable test harnesses and deterministic fixtures

Keep data files outside the Java package tree. The final ZIP must contain source files, data, diagrams, README, and evidence records, but no class files, IDE caches, or operating-system metadata.

Testing and Debugging Plan

Create an executable test harness or focused application test methods. Each test must name the requirement, arrange controlled data, perform one behavior, and compare the actual result with an explicit expected result. Repeat concurrency tests with recreated Thread objects and assert final invariants after join.

Test	Controlled condition	Required evidence
Valid rental	Eligible vehicle and valid period	One active rental and unavailable vehicle
Double booking	Second request for rented vehicle	RentalException and no duplicate active rental
Pricing strategies	Same period under different policies	Correct transparent quote per policy
Maintenance block	Vehicle marked maintenance due	Rental rejected without state change
Ordering	Equal rates or mileage	Stable ID tie breaker
Concurrent requests	Two threads request one vehicle	Exactly one success and consistent final state

Required Debugging Record

- One input or file-format defect and its validation correction.
- One collection, ordering, generic, iterator, lambda, or Stream defect and its correction.
- One lifecycle, interruption, race, visibility, or synchronization defect and its correction.
- For each defect, record the symptom, hypothesis, evidence, root cause, correction, and regression test.

Development and Team Integration Plan

Stage	Team evidence
1 Analyze	Problem statement, actors, invariants, seed data, risks, and definition of done
2 Design	Class and sequence diagrams, package map, collection choices, generic relationships, and concurrency state view
3 Build foundation	Models, exceptions, repositories, file loading, and one end-to-end console workflow
4 Integrate modules	Ordering, iterable aggregate, generics, Streams, annotation inspection, and finite worker behavior

Stage	Team evidence
5 Verify	Normal, boundary, invalid, empty, interruption, and repeated concurrency tests plus defect record
6 Release	Clean source-only ZIP, README, evidence matrix, presentation, and full-team rehearsal

Suggested responsibility areas include domain model, application services, reporting and data access, and verification and integration. These are coordination roles only. Every member must review another area, contribute to integration and testing, and understand the complete project.

Deliverables and Presentation

Deliverable	Contents
Java source package	Complete runnable application, package structure, data files, and no compiled class files
README and design	Build and run instructions, assumptions, limitations, UML class diagram, sequence diagram, and package map
Module evidence matrix	Modules 1 through 9 mapped to classes, methods, tests, and presentation evidence
Testing and debugging record	Expected and actual results, repeated concurrency evidence, three corrected defects, and remaining risks
Contribution record	Member responsibilities, reviews, integration work, and shared testing evidence
Responsible AI record	Purpose, prompt or question, output decision, validation, attribution, and explanation for each material AI contribution
Presentation	10 to 12 minutes, every member speaks, live success and failure demonstrations, design and testing explanation, and instructor questions

Responsible AI Requirements

AI may support clarification, compiler-error interpretation, alternative-design comparison, test generation, API comparison, review, and documentation. It may not replace the team's analysis, implementation ownership, testing, or explanation. Every retained suggestion must be adapted to the project, compiled, tested, and defended by the team. Undocumented AI-generated code or explanations presented without understanding are not acceptable evidence.

Project Reflection

1. Why is PricingPolicy preferable to conditionals inside RentalService?
2. Which rental data should be immutable after checkout?
3. Where must synchronization begin and end to prevent double booking?
4. Which required technique produced the clearest design improvement, and which technique required the most careful justification?
5. What failure case changed your design after testing?
6. What would you improve next without expanding beyond the approved course scope?

EFREI Assessment Governance

This final application and its presentation are mandatory instructional practical work. They consolidate course learning but do not carry an individual on-site grade. The approved assessment model remains unchanged: the Concordia on-site Mid-Term is 50 percent, and EFREI administers and assesses the remaining 50 percent through its final written examination after students return.

The project must therefore produce visible learning evidence without presenting a local percentage rubric. Feedback addresses completeness, correctness, design quality, testing, explanation, teamwork, and readiness for the EFREI examination.

Final Readiness Check

- The complete application compiles and runs from a clean source-only copy using the documented Java version.
- Every Module 1 through Module 9 row points to runnable, tested, and explainable evidence.
- All workers terminate normally or cooperatively after interruption, and the application does not depend on timing luck.
- The team can demonstrate one success, one rejected invalid operation, one report, and one stable concurrency invariant.
- Every member can explain the architecture, core domain rule, test evidence, contribution record, and Responsible AI use.